

A Comparative Study of Monolithic and Modular Architectures in a Dataset Driven Case Tracking Web Application

Mireya Francesca Vella

*Institute of Information and Communication Technology
Malta College of Arts, Science and Technology
Paola, Malta
mireya.vella.i85114@mcast.edu.mt*

Abstract—This document is a model and instructions for L^AT_EX. This and the IEEEtran.cls file define the components of your paper [title, text, heads, etc.]. *CRITICAL: Do Not Use Symbols, Special Characters, Footnotes, or Math in Paper Title or Abstract.

Index Terms—component, formatting, style, styling, insert

I. INTRODUCTION

A. Description of Theme and Topic Rationale

Backend architecture refers to how a server-side system is organised into components, responsibilities, and dependency boundaries [1]. A common architectural choice in backend development is whether to build a monolithic system, in which most functionality is implemented within one tightly connected codebase, or a modular or layered system, in which responsibilities are separated into clearer units such as routing, services, and data access [1]. This choice is important because architecture affects how easily a system can be changed, tested, and maintained over time. Research on architectural smells and technical debt shows that unclear boundaries and poor structure can increase long-term effort and reduce maintainability, supporting the need for evidence-based evaluation of architectural decisions [1]–[3].

This study focuses on a dataset-driven case-tracking application in order to compare these approaches in a controlled manner [5]. Two backends are built with the same user-facing behaviour, including importing a synthetic JSON dataset into PostgreSQL, validating and normalising records, logging import runs, and exposing search and trend functionality [5]. Comparing two implementations with identical requirements is appropriate because it allows differences in maintainability, testability, and operational performance to be measured under consistent conditions rather than assumed [3], [12].

B. Positioning and Research Onion

This study adopts a pragmatist philosophy because it aims to develop and evaluate a working software artefact and to draw conclusions from practical, measurable evidence. This position is appropriate because the outcome is assessed using objective metrics collected from the implemented systems

rather than the researcher’s personal opinion. The study follows a deductive approach, as it seeks to test the predefined hypothesis that a modular or layered architecture provides better maintainability and testability than a monolithic design.

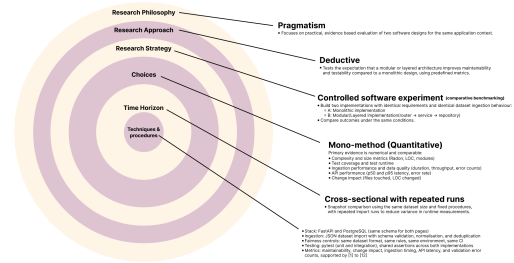


Fig. 1. Research Onion for the Study

The research strategy is a controlled software experiment based on comparative benchmarking. Two backend implementations are developed with identical requirements, identical dataset ingestion behaviour, and the same API contract. The analysis uses a mono-method quantitative strategy, since the research questions are addressed through numerical measures that can be compared across both architectures, including structural metrics, testing results, and performance timings. The time horizon is cross-sectional with repeated runs, meaning that both systems are evaluated at the same stage of completion while measurements are repeated to reduce random variation in runtime results. Recent research also shows growing interest in modular monolith designs and modular boundary practices, while emphasising the need for clearer evaluation evidence across different contexts [9], [10].

C. Background to this Research Theme

Architectural quality is closely related to maintainability, and architectural smells are recurring patterns that often indicate deeper design problems. Systematic evidence shows increasing interest in detecting and managing such smells as part of maintaining long-term system quality [1], while technical debt research also suggests that weak boundaries and widespread dependencies can create future cost [2]. Empirical

studies further link the presence of smells to maintainability outcomes, indicating that architecture can affect measurable software quality rather than simply reflecting design preference [3].

D. Hypothesis, Research Aim, and Purpose Statement

This study will test the hypothesis that a modular or layered architecture is more maintainable and testable than a monolithic architecture.

The purpose of this controlled comparative experiment will be to test the theory that architectural modularity improves software quality by relating backend architecture type to maintainability and testability outcomes, while controlling for identical requirements, dataset, database schema, API contract, test suite, and runtime environment across both implementations. The independent variable will be backend architecture type, defined as a monolithic backend or a modular or layered backend. The dependent variables will be maintainability and testability, defined using tool-based structural measures and automated test evidence.

II. LITERATURE REVIEW

This section reviews the methodologies adopted in recent studies related to software architecture quality and modularity, with a focus on how researchers evaluate architectural smells, technical debt, maintainability, and remodularisation. To provide a structured and comparative lens, this review follows the logic of the research onion by considering research philosophy, methodological approach, strategy, time horizon, and evaluation procedures used to validate architecture-related hypotheses [1], [9].

Across the reviewed works, a largely positivist stance is evident, prioritising objective and measurable outcomes derived from software artefacts such as source code, repository history, and static analysis outputs [1], [3], [6]. Most studies follow a deductive approach by testing expectations such as whether architectural smells are associated with maintainability indicators, whether smell evolution can be observed across versions, or whether modularity can be captured through stronger quantitative metrics [3], [6], [8], [11]. The dominant strategies are empirical mining studies and tool-supported analyses, complemented by evidence synthesis through mapping studies and systematic reviews [1], [9]. Time horizons are commonly cross-sectional (single snapshot of systems) or version-based (repeated measures across releases) [6].

All sources included in this review are academic publications, including peer-reviewed journal articles and peer-reviewed conference proceedings [1]–[12]. Non-academic sources such as vendor documentation, architecture blogs, and practitioner case posts can support implementation decisions, but they are not treated as primary evidence here because they often do not report controlled variables, reproducible methods, or peer-reviewed evaluation.

A. Methodologies of Similar Studies

[1] conducts a systematic mapping study on architectural smell detection. The methodology applies structured search

and selection procedures, then categorises studies by detection approaches, data sources, tool support, and validation methods. This is strong for breadth and transparency, and it supports positioning by showing that detection practices are diverse and not fully standardised. However, mapping studies do not directly test outcomes such as improved maintainability, so they contribute classification and gap identification rather than causal evidence.

[2] proposes an architecture smell knowledge base to support managing architecture technical debt. The contribution is knowledge structuring rather than benchmarking performance. Methodologically, it supports consistent definitions and links smells to debt concepts and management actions. Its limitation is that it provides limited direct quantitative evidence about whether specific architectures reduce maintainability cost, so it complements empirical studies rather than replacing them.

[3] provides an empirical investigation of how architectural smells relate to software maintainability. The methodology detects smells and compares them against maintainability-related indicators using quantitative analysis. A key strength is the use of measurable variables and repeatable pipelines. A limitation is that repository-based evidence is often correlational, meaning maintainability differences may be associated with smells while still being influenced by other factors such as team practices and feature churn.

[6] examines evolution patterns of software-architecture smells across intra-version and inter-version settings. The methodology relies on repeated measurements across releases, which strengthens the evidence by moving beyond single-snapshot analysis. This supports trend-based claims about accumulation and persistence of smells. However, many influences change between versions, so the method is better for describing evolution than proving strict cause-and-effect.

[7] explores the relation between co-changes and architectural smells through repository mining. Co-change behaviour is used as a proxy for coupling and change propagation, then analysed alongside smell presence. The strength is that it is grounded in real maintenance activity. The limitation is that commit granularity and workflow conventions can distort co-change patterns, which can threaten internal validity if not carefully controlled.

[8] proposes M-score as an empirically derived modularity metric. The methodology focuses on improving measurement quality, which is important because modularity is often used as a dependent variable in architecture evaluation. Stronger metrics support clearer comparison across systems. A limitation is that modularity is multi-dimensional, so M-score should be interpreted alongside complementary indicators such as complexity, test outcomes, and change impact.

[11] investigates correlations between architectural smells and static analysis warnings. Methodologically, this triangulates architecture-level detection with another measurement source (static analysis), which can improve construct validity by showing alignment between different quality signals. However, correlations must be interpreted cautiously because static warnings vary in severity and may not uniformly reflect

architectural structure.

[12] presents a search-based remodularisation case study at Adyen. Case study methodology supports practical relevance by evaluating restructuring in a realistic industrial setting. This provides strong context and feasibility evidence, but generalisability is reduced because results can depend on the organisation, codebase constraints, and local engineering practices.

[4] proposes an effort estimation approach for clustering-based remodularisation in a conference companion venue. It supports framing restructuring cost and feasibility, but it is lighter empirical evidence compared to full-length journal studies. Similarly, [10] discusses modular monolith as a trend in a workshop context, supporting conceptual positioning rather than providing strong controlled evaluation.

[9] provides a systematic literature review on modular monolith architecture in cloud environments. This supports positioning by synthesising how modular monolith is defined and evaluated and highlights variation in metrics and methods. Its limitation is that review evidence depends on the quality and consistency of the underlying studies, which further motivates careful control in comparative experiments.

Finally, [5] studies data pipeline quality by identifying influencing factors, root causes of data-related issues, and problem areas for developers. This is methodologically relevant to ingestion-based evaluation because it supports including operational and data-quality indicators (for example, validation errors and processing issues) as measurable outcomes within backend benchmarking.

B. Comparisons of Studies

Across the reviewed studies, a consistent emphasis on measurable evidence is visible, but three methodological families emerge, with differences in how strongly they support causal conclusions. Evidence synthesis papers such as [1] and [9] provide structured overviews and highlight gaps, but they do not directly test architecture outcomes in controlled conditions. Repository-based empirical studies such as [3], [6], [7], and [11] provide scalable, tool-supported measurement, yet often produce association evidence due to limited control of confounding variables. Case study and search-based remodularisation research such as [12] provides detailed, realistic evaluation but can be difficult to generalise.

These differences matter for the present research because they justify a controlled comparative benchmarking design. By holding requirements, dataset ingestion rules, schema, API behaviour, and test procedures constant, stronger internal validity can be achieved than in observational mining studies, while still producing practical evidence relevant to real backend development.

TABLE I
COMPARATIVE OVERVIEW OF STUDIES

Study	Focus	Strategy & Time Horizon	Evidence Type
[1]	Smell detection research landscape	Systematic mapping, cross-sectional	Academic synthesis
[3]	Smells and maintainability relation	Empirical study, cross-sectional (observational)	Repository / code metrics
[6]	Smell evolution patterns	Empirical study, multi-version measurement	Repository / release metrics
[7]	Co-change and smells	Empirical study, version history mining	Co-change (evolution) metrics
[8]	Modularity measurement (M-score)	Empirical metric derivation and evaluation	Modularity metric validation
[9]	Modular monolith in cloud	Systematic literature review, cross-sectional	Academic synthesis
[11]	Smells and static warnings	Empirical study, cross-sectional	Smell detection + static analysis outputs
[12]	Search-based remodularisation	Industrial case study, cross-sectional	Applied evaluation (industrial evidence)

TABLE II
RESEARCH DESIGN AND EVALUATION TOOLS

Study	Research Design	Data Collection	Evaluation Tools / Measures
[1]	Secondary research (mapping)	Study selection and classification	Taxonomies of methods, tool support, evaluation approaches
[3]	Quantitative empirical (observational)	Smell detection + maintainability-related measures	Statistical analysis linking smell presence to maintainability evidence
[6]	Quantitative empirical (multi-version)	Smell detection across releases	Trend, persistence, and evolution analysis across versions
[7]	Quantitative empirical	Version history mining	Co-change analysis examined in relation to architectural smells
[8]	Quantitative metric study	Software structural data	M-score computation, modularity scoring, and metric evaluation
[9]	Secondary research (SLR)	Study selection and synthesis	Comparison of definitions, evaluation practices, and research gaps
[11]	Quantitative empirical (triangulation)	Smell detection + static analysis warnings	Correlation / triangulation between smell indicators and static analysis warnings
[12]	Industrial case study	System data from a real organisation	Search-based remodularisation outcomes; structural improvement evidence

III. RESEARCH METHODOLOGIES

This study aims to evaluate whether a modular or layered backend architecture improves maintainability and testability compared to a monolithic backend when both implementations deliver identical application behaviour under controlled conditions. The study is applied because it produces two working

software artefacts and evaluates them using objective evidence collected through automated tests, tooling, and repeated benchmark runs. The methodological choices are informed by prior architecture-quality research, which commonly uses tool-supported measurement and repository-based evidence to reason about maintainability, smells, modularity, and change impact [1], [3], [6], [7], [8], [11].

Based on this aim, the following research questions were formed:

RQ1: To what extent does a modular or layered backend architecture improve maintainability compared to a monolithic backend when implementing the same requirements?

RQ2: To what extent does a modular or layered backend architecture improve testability compared to a monolithic backend under the same test suite and contract?

RQ3: How does architecture type affect operational performance for dataset ingestion and API responsiveness under the same environment and workload?

RQ4: How does architecture type affect change impact when implementing a controlled change request on the same codebase features?

To answer these research questions, the following objectives were constructed:

- Implement two backend versions of the same case tracking application with identical features, API contract, database schema, and dataset ingestion behaviour.
- Apply fairness controls by using the same dataset format and rules, the same migrations, the same environment and CI configuration, and the same shared integration tests.
- Collect maintainability and structure evidence using tool-based metrics aligned with modularity and architectural quality research, including modularity and smell-related indicators where applicable [1], [3], [6], [8], [11].
- Collect testability evidence using test coverage, test stability, and test execution outcomes under the same test suite and assertions.
- Collect operational performance evidence through repeated ingestion runs and repeated API benchmark runs under the same conditions, including error counts and timing measures, informed by quality concerns in data pipelines [5].
- Execute a controlled change request in both versions and compare change impact using measurable proxies (files touched, lines changed, and affected modules), grounded in evolution and co-change considerations [6], [7].
- Analyse results using descriptive statistics and comparative analysis to determine whether the observed differences support the hypothesis.

A. Research Design

This study adopts a pragmatist philosophy, as it focuses on evidence-based comparison of two software designs in a realistic development context, with the goal of producing practical conclusions that support architectural decision-making. A

deductive approach is employed because the study begins with a predefined hypothesis: a modular or layered backend will be more maintainable and more testable than a monolithic backend. This hypothesis is grounded in existing research linking architectural quality, smells, modularity, and change coupling to maintainability outcomes [2], [3], [6], [7], [8], [11].

A mono-method quantitative design is used, since the hypothesis and research questions can be answered using measurable evidence such as structural indicators, test outcomes, runtime measures, and change impact proxies. The strategy is a controlled software experiment (comparative benchmarking), where two implementations are evaluated under the same conditions to reduce confounding factors. The time horizon is cross-sectional with repeated runs, meaning the comparison is conducted at a defined stage of completion, while performance measures are repeated to reduce noise in runtime results.

B. Environment and Technologies Used

Both backend implementations are developed using FastAPI (Python) with PostgreSQL. The system includes a JSON ingestion pipeline that validates, normalises, deduplicates, and inserts data, while logging import-run summaries. Fairness is maintained by using the same database schema and migrations and by validating identical API behaviour through shared integration tests.

Evaluation is supported through:

- Tool-based structural measurement (e.g., complexity/size indicators and modularity indicators, including M-score where applicable) [8].
- Architecture-quality indicators informed by smell research where feasible [1], [3], [6], [11].
- Automated unit and integration testing with coverage reporting.
- Repeated-run benchmarking for ingestion and API responsiveness, including timing and error reporting.

C. Experiment Design

Two backend artefacts are implemented:

- Architecture A: Monolith - a single backend structure where routing, business rules, and data access are tightly integrated.
- Architecture B: Modular - a backend structure where routing, business rules, and data access are separated into distinct modules with well-defined interfaces.

To strengthen internal validity, both versions are evaluated with the following controls:

- Same API contract and outputs (verified using shared integration tests).
- Same database schema and migrations.
- Same dataset format, validation rules, and ingestion procedure.
- Same environment and CI/testing procedures.

A controlled change request is then implemented in both versions to assess change impact. The change introduces

derived analytics (e.g., completion rate for an optional milestone) and summary statistics (e.g., median/percentiles for milestone duration), enabling measurable comparison aligned with evolution and co-change rationale [6], [7].

D. Reflections on Validity, Reliability, and Generalisability

Internal validity is strengthened through fairness controls, reducing the likelihood that differences are caused by feature drift or environmental variation, which is a common limitation in observational architecture research [3], [6], [11]. Construct validity is supported by using multiple indicators for maintainability and testability, avoiding reliance on a single metric to represent complex software qualities, which aligns with concerns in modularity measurement [8]. Reliability is supported through automation, repeatable measurement pipelines, and repeated performance runs under consistent conditions. Generalisability is limited because the study evaluates one application domain and one technology stack, but transferability is supported by clear documentation of procedures and metrics, consistent with how industrial and case-based evidence is typically contextualised [12].

E. Ethical Considerations

This study does not involve human participants. The dataset is synthetic, reducing the risk of handling personal or sensitive data. Ethical considerations primarily relate to research integrity: methodology, controls, and limitations will be reported transparently, and tool limitations will be acknowledged to avoid overstating conclusions [1], [8]. Results will be presented objectively, including outcomes that do not support the hypothesis.

REFERENCES

- [1] H. Mumtaz, P. Singh, and K. Blincoe, "A Systematic Mapping Study on Architectural Smells Detection," *Journal of Systems and Software*, vol. 173, Art. no. 110885, 2021, doi: 10.1016/j.jss.2020.110885.
- [2] P. Rachow and M. Riebisch, "An Architecture Smell Knowledge Base for Managing Architecture Technical Debt," in *Proc. IEEE/ACM Int. Conf. on Technical Debt (TechDebt)*, pp. 1–10, 2022, doi: 10.1145/3524843.3528092.
- [3] R. Jolak, S. Karlsson, and F. Dobsław, "An Empirical Investigation of the Impact of Architectural Smells on Software Maintainability," *Journal of Systems and Software*, vol. 225, Art. no. 112382, 2025, doi: 10.1016/j.jss.2025.112382.
- [4] A. J. J. Tan, C. Y. Chong, and A. Aleti, "Closing the Loop for Software Remodularisation: REARRANGE, An Effort Estimation Approach for Software Clustering Based Remodularisation," in *Companion Proc. IEEE/ACM Int. Conf. on Software Engineering (ICSE-Companion)*, 2023, pp. 326–327, doi: 10.1109/ICSE-Companion58688.2023.00090.
- [5] H. Foidl, V. Golendukhina, R. Ramler, and M. Felderer, "Data pipeline quality: Influencing factors, root causes of data related issues, and processing problem areas for developers," *Journal of Systems and Software*, vol. 207, Art. no. 111855, Jan. 2024, doi: 10.1016/j.jss.2023.111855.
- [6] P. Gnoyke, S. Schulze, and J. Krüger, "Evolution Patterns of Software-Architecture Smells: An Empirical Study of Intra- and Inter-Version Smells," *Journal of Systems and Software*, vol. 217, Art. no. 112170, 2024, doi: 10.1016/j.jss.2024.112170.
- [7] D. Sas, P. Avgeriou, R. Kruizinga, and R. Scheedler, "Exploring the Relation Between Co-changes and Architectural Smells," *SN Computer Science*, 2021, doi: 10.1007/s42979-020-00407-5.
- [8] E. Pisch, Y. Cai, R. Kazman, J. Lefever, and H. Fang, "M-score: An Empirically Derived Software Modularity Metric," in *Proc. ACM/IEEE Int. Symp. on Empirical Software Engineering and Measurement (ESEM)*, pp. 382–392, 2024, doi: 10.1145/3674805.3686697.
- [9] L. F. Al-Qora'n and A. Al-Said Ahmad, "Modular Monolith Architecture in Cloud Environments: A Systematic Literature Review," *Future Internet*, vol. 17, no. 11, Art. no. 496, 2025, doi: 10.3390/fi17110496.
- [10] R. Su and X. Li, "Modular Monolith: Is This the Trend in Software Architecture?," in *Proc. 1st Int. Workshop on New Trends in Software Architecture (SATrends '24)*, 2024, doi: 10.1145/3643657.3643911.
- [11] M. Esposito, M. Robredo, F. Arcelli Fontana, and V. Lenarduzzi, "On the Correlation Between Architectural Smells and Static Analysis Warnings," *Software Quality Journal*, 2025, doi: 10.1007/s11219-025-09730-7.
- [12] C. Schröder, A. van der Feltz, A. Panichella, and M. Aniche, "Search Based Software Re Modularization: A Case Study at Adyen," in *Proc. IEEE/ACM 43rd Int. Conf. on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, 2021, pp. 81–90, doi: 10.1109/ICSE-SEIP52600.2021.00017.